



Social Media Processing

Question 1

Build and evaluate two regression models for humour rating prediction (a) without special preprocessing applied and (b) with special preprocessing applied to the training data (Tasks 1 and 2 in the script). In your report, include the output of the `print(model)` command to show your model structure and training epochs (with loss values), as well as the mean squared error (MSE) scores over the test set for both models. Plot the histograms of the predicted and true humour ratings for the first model (a) and comment on the results. Describe the differences in performance between the two models (a and b) and discuss why they occur.

output of the `print(model)`:

```
[13] print(model)
BERTRegressionModel(
  (bert): DistilBertModel(
    (embeddings): Embeddings(
      (word_embeddings): Embedding(30522, 768, padding_idx=0)
      (position_embeddings): Embedding(512, 768)
      (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
      (dropout): Dropout(p=0.1, inplace=False)
    )
    (transformer): Transformer(
      (layer): ModuleList(
        (0-5): 6 x TransformerBlock(
          (attention): DistilBertSelfAttention(
            (q_lin): Linear(in_features=768, out_features=768, bias=True)
            (k_lin): Linear(in_features=768, out_features=768, bias=True)
            (v_lin): Linear(in_features=768, out_features=768, bias=True)
            (out_lin): Linear(in_features=768, out_features=768, bias=True)
            (dropout): Dropout(p=0.1, inplace=False)
          )
          (sa_layer_norm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
          (ffn): FFN(
            (dropout): Dropout(p=0.1, inplace=False)
            (lin1): Linear(in_features=768, out_features=3072, bias=True)
            (lin2): Linear(in_features=3072, out_features=768, bias=True)
            (activation): GELUActivation()
          )
          (output_layer_norm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
        )
      )
    )
    (global_avg_pool): GlobalAveragePooling1DMasked()
    (hidden_layer): Linear(in_features=768, out_features=16, bias=True)
    (activation): Sigmoid()
    (output_reg): Linear(in_features=16, out_features=1, bias=True)
  )
)
```

Test and MSE Loss without special preprocessing:

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
WARNING: huggingface_hub.utils._http:warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

Loading weights: 100% 100/100 [00:00<00:00, 200.02it/s, Materializing param=transformer.layer.5.sa_layer_norm.weight]
DistilBertModel LOAD REPORT from: distilbert-base-uncased
Key | Status |
-----|-----|
vocab_projector.bias | UNEXPECTED |
vocab_layer_norm.bias | UNEXPECTED |
vocab_transform.bias | UNEXPECTED |
vocab_transform.weight | UNEXPECTED |
vocab_layer_norm.weight | UNEXPECTED |

Notes:
- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
Epoch 1/5: 100% 20/20 [00:43<00:00, 2.17s/it]
Train Loss: 0.0123, Val Loss: 0.0115
Epoch 2/5: 100% 20/20 [00:46<00:00, 2.30s/it]
Train Loss: 0.0110, Val Loss: 0.0107
Epoch 3/5: 100% 20/20 [00:48<00:00, 2.45s/it]
Train Loss: 0.0097, Val Loss: 0.0102
Epoch 4/5: 100% 20/20 [00:47<00:00, 2.40s/it]
Train Loss: 0.0086, Val Loss: 0.0106
Epoch 5/5: 100% 20/20 [00:48<00:00, 2.41s/it]
Train Loss: 0.0074, Val Loss: 0.0107

preds, test_targets = evaluate_model(model, test_loader, device)

Test loss: 0.0130
Test Mean Squared Error: 0.0136
```

Get the array of predictions here so that you can plot the outputs later.

Test and MSE Loss with special preprocessing:

```
model = BERTRegressionModel()
train_model(model, train_loader, dev_loader, device)
```

Loading weights: 100% 100/100 [00:00<00:00, 225.90it/s, Materializing param=transformer.layer.5.sa_layer_norm.weight]
DistilBertModel LOAD REPORT from: distilbert-base-uncased
Key | Status |
-----+-----+
vocab_projector.bias | UNEXPECTED |
vocab_layer_norm.bias | UNEXPECTED |
vocab_transform.bias | UNEXPECTED |
vocab_transform.weight | UNEXPECTED |
vocab_layer_norm.weight | UNEXPECTED |

Notes:
- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
Epoch 1/5: 100% | 20/20 [00:49<00:00, 2.47s/it]
Train Loss: 0.0136, Val Loss: 0.0121
Epoch 2/5: 100% | 20/20 [00:47<00:00, 2.38s/it]
Train Loss: 0.0117, Val Loss: 0.0115
Epoch 3/5: 100% | 20/20 [00:48<00:00, 2.43s/it]
Train Loss: 0.0108, Val Loss: 0.0110
Epoch 4/5: 100% | 20/20 [00:48<00:00, 2.42s/it]
Train Loss: 0.0099, Val Loss: 0.0109
Epoch 5/5: 100% | 20/20 [00:48<00:00, 2.43s/it]
Train Loss: 0.0090, Val Loss: 0.0106

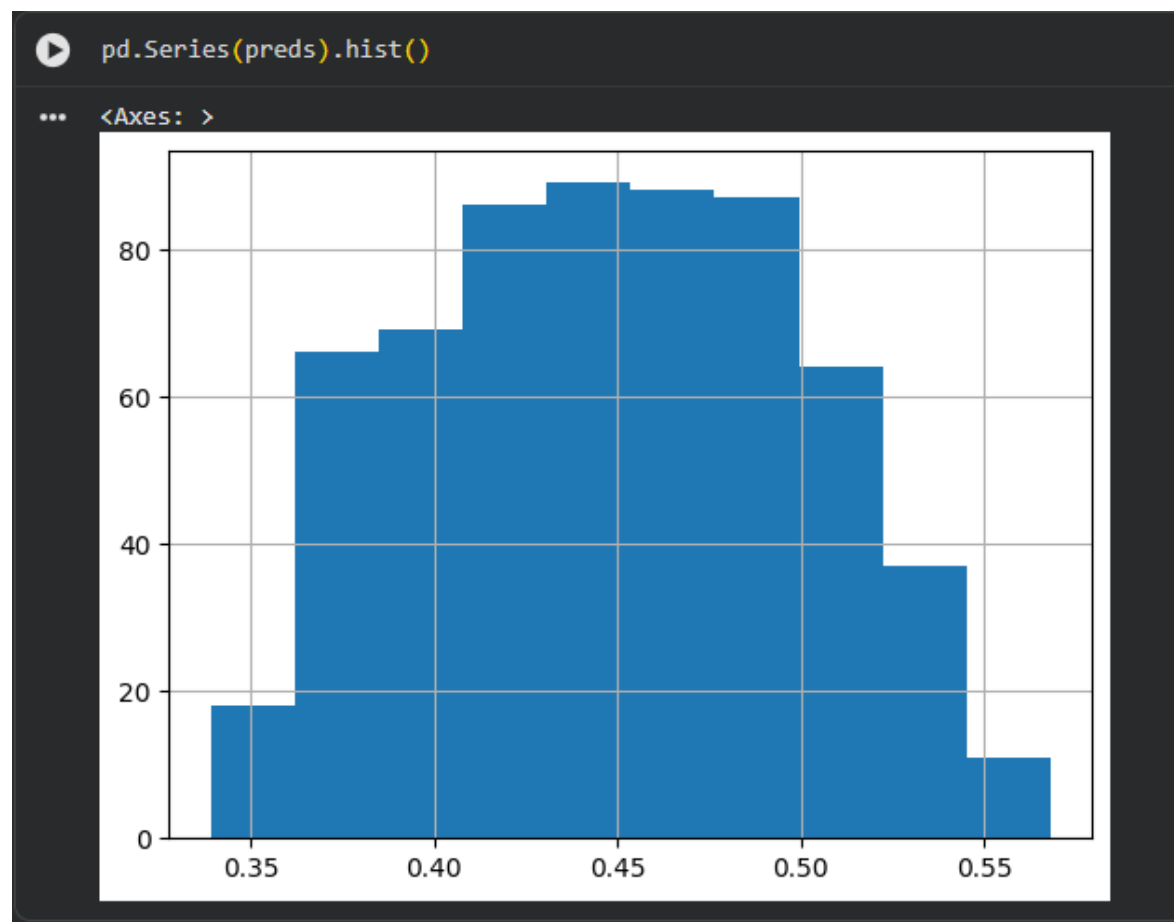
```
preds, test_targets = evaluate_model(model, test_loader, device)
```

Test loss: 0.0124
Test Mean Squared Error: 0.0128

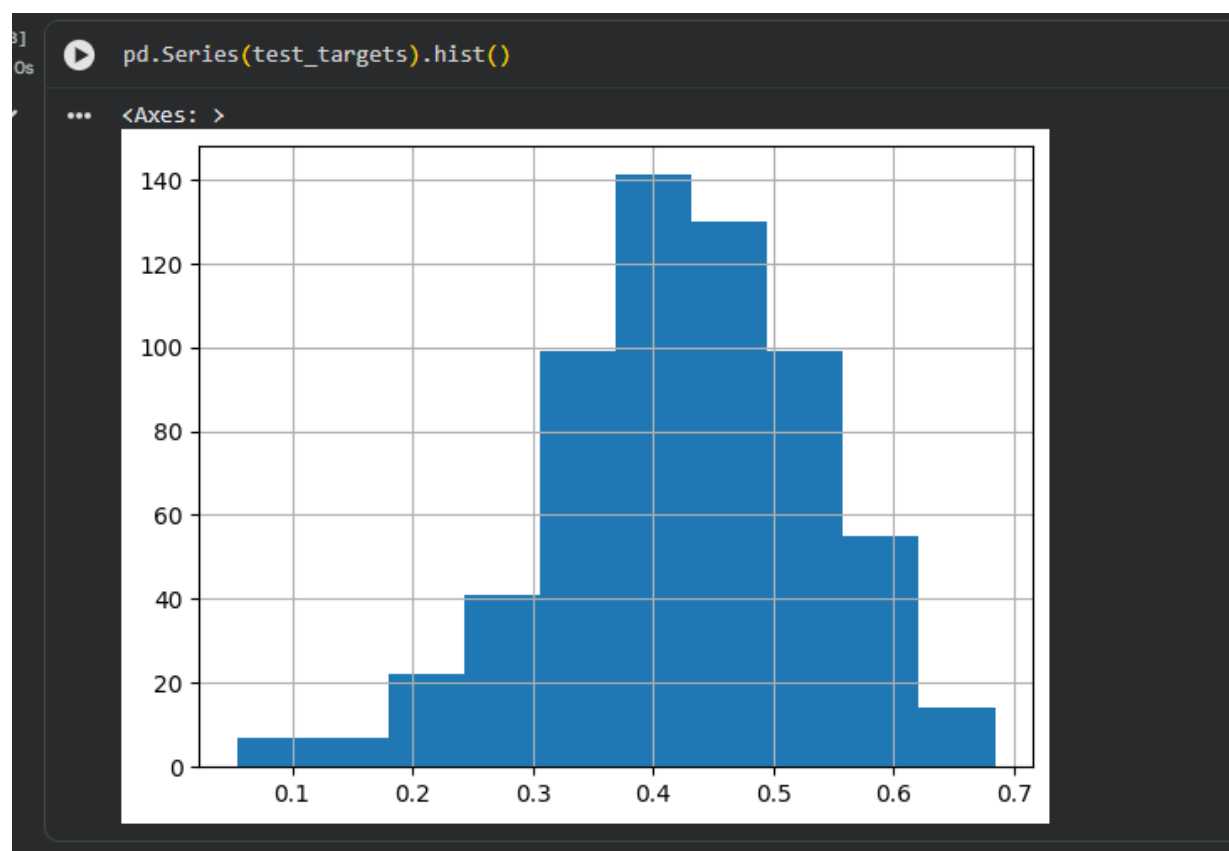
Comparison Between both models:

The model trained on Ekphrasis preprocessed text achieved a lower MSE loss as compared to the model trained on raw text. This indicates that there is a minor improvement in humor rating prediction after preprocessing which is likely because of Ekphrasis reduces linguistic noise in social media posts by normalizing contractions, Punctuations and other irregular forms. However the improvement is relatively small which suggests that preprocessing can sometimes remove stylistic cues that are useful for humor detection. In our experiment it seems that it has reduced noise more than it removed useful information.

Histogram of predicted humour rating:



Histogram of true humour rating:



The input range of predicted humour rating is between (0.3 to 0.6) the model is predicting moderate humour values while for the true ratings it is much wider around 0.05 to 0.7. The true data includes very low, very high and moderate values. The model is preferring safe predictions because it has to minimize mean squared error. MSE punishes large errors so the model avoids strong predictions.

Question 2

Augment the training data twice for the regression model (Task 1 above) with the two following methods (a) synonym replacement from Wordnet and (b) deletion of random words (Task 3). In your report, include the output of the training epochs (with loss values), as well as the MSE scores over the test set for both models. Describe the differences in performance between the models (a) and (b) and hypothesise why they occur.

Synonym Replacement

```
Notes:
- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical
Epoch 1/5: 100%|██████████| 39/39 [01:33<00:00, 2.40s/it]
Train Loss: 0.0118, Val Loss: 0.0110
Epoch 2/5: 100%|██████████| 39/39 [01:33<00:00, 2.40s/it]
Train Loss: 0.0101, Val Loss: 0.0105
Epoch 3/5: 100%|██████████| 39/39 [01:33<00:00, 2.40s/it]
Train Loss: 0.0085, Val Loss: 0.0107
Epoch 4/5: 100%|██████████| 39/39 [01:33<00:00, 2.39s/it]
Train Loss: 0.0067, Val Loss: 0.0108
Epoch 5/5: 100%|██████████| 39/39 [01:33<00:00, 2.39s/it]
Train Loss: 0.0052, Val Loss: 0.0118
```

```
35] preds, test_targets = evaluate_model(model, test_loader, device)
✓ 2s
```

```
... Test loss: 0.0137
Test Mean Squared Error: 0.0141
```

Random Deletion

```
34] model = BERTRegressionModel()
✓ 8m train_model(model, train_loader, dev_loader, device)
```

```
Loading weights: 100% ██████████ 100/100 [00:00<00:00, 243.54it/s, Materializing param=transformer.layer.5.sa_layer_norm.weight]
DistilBertModel LOAD REPORT from: distilbert-base-uncased
Key | Status |
-----+-----+
vocab_projector.bias | UNEXPECTED |
vocab_layer_norm.bias | UNEXPECTED |
vocab_transform.bias | UNEXPECTED |
vocab_transform.weight | UNEXPECTED |
vocab_layer_norm.weight | UNEXPECTED |

Notes:
- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
Epoch 1/5: 100%|██████████| 39/39 [01:36<00:00, 2.48s/it]
Train Loss: 0.0118, Val Loss: 0.0110
Epoch 2/5: 100%|██████████| 39/39 [01:36<00:00, 2.48s/it]
Train Loss: 0.0102, Val Loss: 0.0105
Epoch 3/5: 100%|██████████| 39/39 [01:36<00:00, 2.47s/it]
Train Loss: 0.0087, Val Loss: 0.0106
Epoch 4/5: 100%|██████████| 39/39 [01:36<00:00, 2.48s/it]
Train Loss: 0.0070, Val Loss: 0.0110
Epoch 5/5: 100%|██████████| 39/39 [01:36<00:00, 2.47s/it]
Train Loss: 0.0053, Val Loss: 0.0114
```

```
35] preds, test_targets = evaluate_model(model, test_loader, device)
✓ 2s
```

```
... Test loss: 0.0129
Test Mean Squared Error: 0.0133
```

The model trained on random deletion performed slightly better than the model trained on synonym replacement. One possible reason could be that synonym replacement from word net may replace words that are semantically not natural or kill the context of the sentence. This can change the original humor which leads to less improvement. In contrast, random deletion can act like a dropout layer for

Bert which forces it to focus on the remaining word to better understand the context but excessive deletion can remove important humorous words so we have to be careful.

Question 3

One way to improve the performance is to perform ensembling of three regressors. Try this (Task 4) and see if it improves the performance of the model from Task 1. Use non-augmented data. In your report, include the output of the training epochs (with loss values), as well as the MSE scores over the test set. Describe the difference in performance to the model from Task 1.

First Run of Ensembling :

```
Notes:
- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
Epoch 1/5: 100%|██████████| 20/20 [00:49<00:00, 2.49s/it]
Train Loss: 0.0121, Val Loss: 0.0117
Epoch 2/5: 100%|██████████| 20/20 [00:48<00:00, 2.41s/it]
Train Loss: 0.0108, Val Loss: 0.0107
Epoch 3/5: 100%|██████████| 20/20 [00:48<00:00, 2.41s/it]
Train Loss: 0.0098, Val Loss: 0.0106
Epoch 4/5: 100%|██████████| 20/20 [00:48<00:00, 2.42s/it]
Train Loss: 0.0088, Val Loss: 0.0105
Epoch 5/5: 100%|██████████| 20/20 [00:48<00:00, 2.41s/it]
Train Loss: 0.0081, Val Loss: 0.0105
Test loss: 0.0120
Test Mean Squared Error: 0.0124
Loading weights: 100%|██████████| 100/100 [00:00<00:00, 288.38it/s, Materializing param=transformer.layer.5.sa_layer_norm.weight]
DistilBertModel LOAD REPORT from: distilbert-base-uncased
..
```

Second Run:

```
Notes:
- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
Epoch 1/5: 100%|██████████| 20/20 [00:48<00:00, 2.43s/it]
Train Loss: 0.0123, Val Loss: 0.0117
Epoch 2/5: 100%|██████████| 20/20 [00:48<00:00, 2.41s/it]
Train Loss: 0.0111, Val Loss: 0.0108
Epoch 3/5: 100%|██████████| 20/20 [00:48<00:00, 2.41s/it]
Train Loss: 0.0102, Val Loss: 0.0106
Epoch 4/5: 100%|██████████| 20/20 [00:48<00:00, 2.42s/it]
Train Loss: 0.0090, Val Loss: 0.0106
Epoch 5/5: 100%|██████████| 20/20 [00:48<00:00, 2.42s/it]
Train Loss: 0.0080, Val Loss: 0.0106
Test loss: 0.0129
Test Mean Squared Error: 0.0135
Loading weights: 100%|██████████| 100/100 [00:00<00:00, 451.93it/s, Materializing param=transformer.layer.5.sa_layer_norm.weight]
```

Third Run:

```
Notes:
- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
Epoch 1/5: 100%|██████████| 20/20 [00:48<00:00, 2.42s/it]
Train Loss: 0.0131, Val Loss: 0.0122
Epoch 2/5: 100%|██████████| 20/20 [00:48<00:00, 2.42s/it]
Train Loss: 0.0115, Val Loss: 0.0113
Epoch 3/5: 100%|██████████| 20/20 [00:48<00:00, 2.42s/it]
Train Loss: 0.0106, Val Loss: 0.0106
Epoch 4/5: 100%|██████████| 20/20 [00:48<00:00, 2.42s/it]
Train Loss: 0.0093, Val Loss: 0.0104
Epoch 5/5: 100%|██████████| 20/20 [00:48<00:00, 2.42s/it]
Train Loss: 0.0083, Val Loss: 0.0108
Test loss: 0.0129
Test Mean Squared Error: 0.0136
```

Overall MSE:

```
[40]
✓ 0s from sklearn.metrics import classification_report, mean_squared_error

mean_preds = np.mean(np.array(all_model_preds), axis=0)
ensemble_mse = mean_squared_error(test_targets, mean_preds)

print('Ensemble Test MSE : {:.4f}'.format(ensemble_mse))

*** Ensemble Test MSE : 0.0129
```

Task 1 output:

```
*** Test loss: 0.0130
    Test Mean Squared Error: 0.0136
```

Comparison with the model of task 1:

The ensemble model achieved a better performance than the baseline model of task 1. The gain is small but consistent because ensembling tries to reduce prediction variance and smooth errors by individual models.

Question 4

Build and evaluate the multi-task learning regressor (Task 5) trained simultaneously with the second regression task of offense rating and In your report, include the output training epochs (with loss values), as well as the MSE scores over the test set for multi-task models. Compare the results with the base model from Task 1, describe the differences in performance between the two models and discuss why they occur

The Multi Task Regressor Model:

```
[57]
✓ 4m # Instantiate model and train
    train_model(model, train_loader, dev_loader, device)

*** Epoch 1/5: 100%|██████████| 20/20 [00:49<00:00, 2.48s/it]
    Train Loss: 0.1041, Val Loss: 0.0780
    Epoch 2/5: 100%|██████████| 20/20 [00:47<00:00, 2.40s/it]
    Train Loss: 0.0701, Val Loss: 0.0682
    Epoch 3/5: 100%|██████████| 20/20 [00:48<00:00, 2.40s/it]
    Train Loss: 0.0641, Val Loss: 0.0660
    Epoch 4/5: 100%|██████████| 20/20 [00:48<00:00, 2.41s/it]
    Train Loss: 0.0627, Val Loss: 0.0650
    Epoch 5/5: 100%|██████████| 20/20 [00:48<00:00, 2.40s/it]
    Train Loss: 0.0615, Val Loss: 0.0643

[58]
✓ 2s preds, test_targets = evaluate_model(model, test_loader, device, multi_task=True)

    Test loss: 0.0127
    Test Mean Squared Error: 0.0133
```

Task1 Model:

```
*** Test loss: 0.0130
    Test Mean Squared Error: 0.0136
```

Comparison of both Models

The multi task model was trained to predict both humour and offense rating simultaneously. During training the model learned representations for both humour and offense prediction. The multi task model achieved a slightly better performance in humour prediction as compared to the base model. This improvement can be because of some linguistic characteristics shared between humor and offense like sarcasm, irony and proactive wording can be associated with offensive content. When learning both tasks simultaneously the shared distil Bert encoder can capture more semantic and stylistic features of the text. The improvement is relatively small which suggests that, although the tasks are related but they are not perfectly aligned.